FINAL PROJECT REPORT

CITY ISSUE REPORTING DASHBOARD

A MERN Stack Web Application

Submitted by

Student Name(s): _____G . Naveen Kumar_____

Roll Number(s): _____25MVCSMR0568_____

Department: _____CSE(AIML)_____

College Name: _____MALLA REDDY TECHNICAL CAMPUS_____

Academic Year

2026–2027

Department / College Project Submission

TABLE OF CONTENTS

1. Cover Page
2. Abstract & Introduction
3. Problem Statement, Objectives & Scope
4. Features & Functional Requirements
5. Algorithms & Methodology
6. Technologies & Implementation
7. Results, Testing & Performance
8. Conclusion, Future Scope & References

Note: Page numbers can be added/updated in Word after final formatting.

SYSTEM WORKFLOW

Citizen / Administrator
React.js User Interface
Express.js REST API + Node.js
Validation and Authorization
MongoDB (Users and Issues)
JSON Response → Updated Dashboard

↓
↓
↓
↓
↓

2. ABSTRACT & INTRODUCTION

2.1 Abstract

The City Issue Reporting Dashboard is a web-based application proposed using the MERN stack: MongoDB, Express.js, React.js, and Node.js. It provides a centralized platform for citizens to report common civic problems such as potholes, garbage accumulation, damaged streetlights, water leakage, and drainage issues. A report can include a title, description, category, location, image, and status. Authorized administrators can review reports, manage issue records, and update their progress. MongoDB stores application records, while REST APIs connect the React frontend to the backend. The project demonstrates full-stack web development, CRUD operations, API design, data validation, and authentication with role-based access where implemented.

2.2 Introduction

Cities regularly face public issues that need to be reported and resolved in a timely manner. Traditional reporting methods—such as phone calls, paper forms, or informal communication—can make it difficult to maintain centralized records and track progress. A web-based reporting dashboard can organize these reports and make their status easier to monitor.

2.3 Problem Background

Without a centralized system, complaints may be recorded inconsistently, duplicate reports can be difficult to identify, and citizens may not know whether an issue is being addressed. Administrators also need a structured way to categorize reports, view locations and images, and update the status of each complaint.

2.4 Proposed Solution

The proposed solution is a MERN-based web application. Citizens can submit issue reports through a React interface. The Express.js and Node.js backend validates requests and performs database operations through MongoDB. Administrators can review reports and update their status. The dashboard can display issue counts and support searching and filtering by category, location, or status.

2.5 Purpose of the Project

The purpose is to simplify civic issue reporting, maintain structured digital records, demonstrate integration of frontend and backend technologies, and provide a foundation for transparent complaint tracking. The application supports the reporting and management workflow; actual municipal integration depends on future development and authorization.

3. PROBLEM STATEMENT, OBJECTIVES & SCOPE

3.1 Problem Statement

Build a MERN-based City Issue Reporting Dashboard that enables citizens to report city problems and allows authorized users to manage issues, categories, locations, images, and statuses using MongoDB. React.js provides the user interface, while Node.js and Express.js provide backend REST APIs. The system supports the main operations required for the project, including adding, viewing, updating, and managing records, along with authentication and role-based access where required.

3.2 Objectives

- Demonstrate full-stack development using the MERN stack.

- Implement CRUD operations using MongoDB.
- Build REST APIs using Node.js and Express.js.
- Develop a user-friendly frontend using React.js.
- Store issue category, location, image reference, and status.
- Implement authentication and role-based access where required.
- Display project information and results clearly.
- Validate user input and test the main application workflows.

3.3 Project Scope

The scope includes user registration and login, submitting civic issue reports, viewing issue details, updating authorized records, deleting records where permitted, managing issue status, and filtering reports. A report may include a title, description, category, location, image URL or file reference, reporting user, status, and timestamps. Features such as GPS mapping, automatic department assignment, SMS notifications, and direct municipal system integration are considered future enhancements unless separately implemented.

3.4 Target Users

Citizens can register and submit complaints and view the status of their reports. Administrators can view and manage reports, update statuses, and monitor overall issue counts. Maintenance staff may be added as a separate role if assignment and resolution workflows are implemented.

4. FEATURES & FUNCTIONAL REQUIREMENTS

4.1 Main Features

- User registration and login.
- Issue reporting with title and description.
- Category selection (for example, roads, sanitation, water, streetlights, and drainage).
- Location entry and optional image upload.
- Issue list and issue-detail views.
- Status tracking: Reported, In Progress, Resolved, or Rejected.
- Search and filtering by status, category, or location.
- Dashboard summary cards and administrative controls.

4.2 User Functionalities

A citizen can create an account, log in, submit a report, and view relevant reports. Depending on the permissions implemented, the citizen may edit or delete their own report. An administrator can view all reports, manage issue records, update statuses, and review dashboard summaries. Every protected operation must be authorized by the backend, not only hidden in the frontend.

4.3 CRUD Operations

Create: submit a new issue using a POST request.

Read: retrieve issue lists or a specific issue using GET requests.

Update: edit issue details using PUT/PATCH requests.

Delete: remove an authorized issue using a DELETE request.

These operations are exposed through Express routes and use Mongoose or the MongoDB driver to interact with MongoDB.

Operation	HTTP Method	Purpose
Create	POST	Add an issue
Read	GET	View issue records
Update	PUT / PATCH	Modify issue details/status
Delete	DELETE	Remove an authorized issue

4.4 Authentication & Validation

Authentication verifies a user's identity. Passwords should be stored as salted hashes using a suitable password-hashing library, never as plain text. Role-based authorization restricts administrator actions. Backend validation should check required fields, allowed status values, valid identifiers, input lengths, and supported image types and sizes. API errors should return appropriate HTTP status codes without exposing secrets or stack traces.

4.5 System Requirements

Hardware: a computer or laptop with at least 4 GB RAM recommended, sufficient storage, and network access where hosted services are used.

Software: Windows, Linux, or macOS; Node.js and npm; React.js; Express.js; MongoDB Community Server or MongoDB Atlas; Visual Studio Code; a modern web browser; and Postman or an equivalent API testing tool.

4.6 REST API Design

POST /api/auth/register — register a user.

POST /api/auth/login — authenticate a user.

POST /api/issues — create an issue.

GET /api/issues — retrieve authorized issue records.

GET /api/issues/:id — retrieve one issue.

PUT /api/issues/:id — update issue details.

PATCH /api/issues/:id/status — update issue status.

DELETE /api/issues/:id — delete an authorized issue.

GET /api/dashboard/stats — retrieve dashboard statistics.

Exact routes may differ according to the final implementation.

5. ALGORITHMS & METHODOLOGY

5.1 System Workflow

1. The user opens the web application.
2. The user registers or logs in if authentication is required.
3. The user fills in the issue form and optionally selects an image.

4. The frontend performs basic validation and sends the request to the backend.
5. The backend checks input data and authorization.
6. The backend creates or retrieves the relevant MongoDB record.
7. The API returns a response to the frontend.
8. The frontend displays the issue or an appropriate success/error message.
9. An authorized administrator can later update the issue status.

5.2 Algorithm: Issue Submission

Input: title, description, category, location, optional image.

10. Start.
11. Read the values entered by the user.
12. Check required fields and validate the values.
13. If validation fails, display an error and stop.
14. Send the report to the issue API.
15. Verify authentication and permissions on the server.
16. Save the validated issue record in MongoDB.
17. Return the created record and a success response.
18. Refresh the issue list or show the new report.
19. Stop.

5.3 Algorithm: Status Update

Input: issue ID and requested status.

20. Start and authenticate the administrator.
21. Check the administrator's role on the server.
22. Validate the issue ID and requested status.
23. Retrieve the matching issue from MongoDB.
24. If the issue does not exist, return a not-found response.
25. Update the status and save the record.
26. Return the updated issue and refresh the interface.
27. Stop.

5.4 Development Methodology

The project can be developed incrementally: (1) requirement analysis, (2) system and database design, (3) backend API development, (4) React frontend development, (5) database and image handling integration, (6) authentication and authorization, (7) testing and debugging, and (8) deployment and documentation. Each stage should be reviewed before proceeding to the next.

5.5 Data Flow

The React frontend sends HTTP requests to the Express API. Express middleware validates requests and checks authentication and authorization. Controllers execute business logic and use Mongoose or the MongoDB driver to read or modify records. The backend returns JSON responses, which React renders as issue cards, tables, messages, and dashboard statistics. If images are uploaded, the application stores the file in configured storage and saves its URL or path in the database.

5.6 Data Model

Issue record fields may include: title (String), description (String), category (String or category ID), location (String or GeoJSON), imageUrl (String), status (String), reportedBy (User ID), createdAt (Date), and updatedAt (Date). A User record may include name, email, passwordHash, role, and timestamps. A separate Category collection can be used if categories need to be managed dynamically.

6. TECHNOLOGIES & IMPLEMENTATION

6.1 Frontend Technologies

React.js is used to create reusable UI components and pages such as Login, Register, Dashboard, Report Issue, Issue List, and Issue Details. HTML provides page structure, CSS provides styling and responsive layouts, and JavaScript handles interactions. Axios or the Fetch API can be used for HTTP requests.

6.2 Backend Technologies

Node.js provides the server-side JavaScript runtime. Express.js defines REST routes and middleware. Controllers process requests, validate input, and return responses. Authentication middleware verifies sessions or tokens, while role-based middleware protects administrative operations. Multer may be used to process multipart file uploads when local or server-side uploads are supported.

6.3 Database

MongoDB stores issue and user records as documents. Mongoose can define schemas, validation rules, and database models. The main collections are users and issues; categories may be stored in a separate collection. Database credentials and authentication secrets should be kept in environment variables and excluded from source control.

6.4 Tools & APIs

Visual Studio Code is used for development; npm manages packages; Postman tests API endpoints; Git and GitHub can track source-code changes; MongoDB Atlas or MongoDB Community Server provides the database. REST APIs carry data between the React client and the Express backend in JSON or multipart form-data format, depending on the operation.

6.5 Implementation Details

The backend connects to MongoDB and exposes endpoints for issue creation, retrieval, updates, deletion, authentication, and dashboard statistics. The frontend presents forms and report lists and calls the endpoints. Dashboard counts should be calculated from database records or a dedicated aggregation query. Search and filters can be implemented through query parameters. Image storage should be configured deliberately: development may use a local uploads folder, while production should use suitable persistent or object storage. The database stores the resulting image path or URL.

6.6 Suggested Folder Structure

city-issue-dashboard/

client/

src/components/

src/pages/

src/services/issueApi.js

src/App.jsx

server/

models/Issue.js

models/User.js

routes/issueRoutes.js

routes/authRoutes.js

controllers/

middleware/

uploads/ (if local uploads are used)

server.js

.env

The structure is a recommendation and should be adjusted to match the actual code.

7. RESULTS, TESTING & PERFORMANCE

7.1 Project Results

The intended result is a web application that allows citizens to submit civic issue reports and authorized administrators to manage and track them. When implemented, the application should store reports in MongoDB, expose CRUD operations through REST APIs, and display the current issue records and status summaries in React. Replace this description with verified results from the completed implementation.

7.2 Test Cases

Test 1 — Registration: submit valid user details; expected result: account is created.

Test 2 — Login: enter correct credentials; expected result: authentication succeeds.

Test 3 — Invalid login: enter incorrect credentials; expected result: access is denied.

Test 4 — Create issue: submit valid issue details; expected result: issue is stored and returned.

Test 5 — Read issues: open the issue list; expected result: authorized records are displayed.

Test 6 — Update issue: edit an allowed field; expected result: changes are saved.

Test 7 — Update status: administrator selects a valid status; expected result: status is updated.

Test 8 — Delete issue: authorized user deletes a permitted record; expected result: record is removed.

Test 9 — Missing required field: submit incomplete data; expected result: validation error is shown.

Test 10 — Unauthorized operation: a non-admin attempts an admin-only action; expected result: request is rejected.

Test 11 — Image upload: upload a supported file; expected result: file is processed and its reference is saved.

Test 12 — Search/filter: choose a category or status; expected result: matching records are shown.

Test	Action	Expected Result
Registration	Submit valid details	Account created
Login	Correct credentials	Login succeeds
Invalid login	Incorrect password	Access denied
Create issue	Submit valid report	Record saved
Read issues	Open issue list	Records displayed
Update issue	Edit permitted field	Changes saved
Status update	Admin changes status	Status updated
Delete issue	Authorized deletion	Record removed
Validation	Omit required field	Error displayed
Authorization	Citizen uses admin endpoint	Request rejected
Image upload	Upload supported image	File processed
Filtering	Filter by category/status	Matching records shown

7.3 Testing Methods

Unit testing checks individual functions and components. Integration testing checks the connection between React, Express, and MongoDB. API testing verifies endpoint methods, responses, validation, and error handling. System testing checks the full workflow. Security testing reviews authentication, authorization, input validation, and file-upload restrictions. Usability testing checks whether users can report and track issues without confusion.

7.4 Performance Analysis

Performance should be evaluated by measuring API response times, page-load time, database query latency, and behavior with multiple records or simultaneous requests. Indexes on frequently filtered fields, pagination, image-size limits, and optimized React rendering can improve responsiveness. Record actual test conditions and measured values before making quantitative performance claims.

7.5 Screenshots / Output

Insert screenshots captured from the running project in the following places:

Figure 7.1 — Login or Registration Page: [Insert screenshot]

Figure 7.2 — Main Dashboard and Summary Cards: [Insert screenshot]

Figure 7.3 — Report Issue Form: [Insert screenshot]

Figure 7.4 — Issue List with Filters: [Insert screenshot]

Figure 7.5 — Issue Details and Status: [Insert screenshot]

Figure 7.6 — Administrator Management View: [Insert screenshot]

Figure 7.7 — MongoDB Sample Record or API Test: [Insert screenshot]

Only include screenshots and results that correspond to features actually implemented.

8. CONCLUSION, FUTURE SCOPE & REFERENCES

8.1 Conclusion

The City Issue Reporting Dashboard applies the MERN stack to the organization of civic complaints. The proposed system combines a React user interface, REST APIs built with Node.js and Express.js, and MongoDB for persistent storage. It is designed to support issue reporting, CRUD operations, category and location information, image references, status tracking, and authentication with role-based access where required. The project provides practical experience in full-stack development, database modeling, API design, validation, and testing. Its effectiveness should be confirmed through implementation and test results.

8.2 Future Enhancements

- Map-based visualization of issue locations using geographic coordinates.
- GPS-based location capture with user permission.
- Email, SMS, or in-app notifications when status changes.
- Automatic assignment of issues to relevant departments.
- Analytics to identify frequently reported issue types and affected areas.
- Multilingual support and improved mobile accessibility.
- Duplicate-report detection and image-assisted categorization.
- Citizen feedback after an issue is resolved.
- Deployment with monitoring, backups, and scalable file storage.
- Integration with municipal systems, subject to approval and API availability.

8.3 References / Documentation

28. MongoDB Documentation — https://www.mongodb.com/docs/
29. React Documentation — https://react.dev/
30. Node.js Documentation — https://nodejs.org/docs/
31. Express.js Documentation — https://expressjs.com/
32. Mongoose Documentation — https://mongoosejs.com/docs/
33. MDN Web Docs — https://developer.mozilla.org/
34. Postman Learning Center — https://learning.postman.com/

These references provide technical documentation for the technologies used in the project.

REPORT COMPLETION NOTE

This report is prepared from the supplied project problem statement. Replace the cover-page placeholders with the actual student, roll number, department, college, and guide details. Add screenshots from the implemented application and record actual testing/performance results before final submission. Do not claim features as implemented unless they are present in the project.